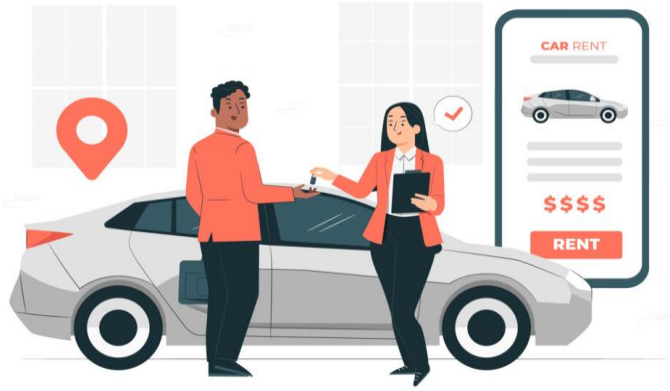
Project name

Car Rental System



Booking of a safe Journey

Name: Neha Jadhav

Batch: T347

Introduction

The **Car Rental System** is developed to systematically manage and monitor the operations of a car rental service, including customer records, vehicle details, booking management, and payment processing. This SQL-based project provides a structured environment that simulates real-world business processes within the car rental industry.

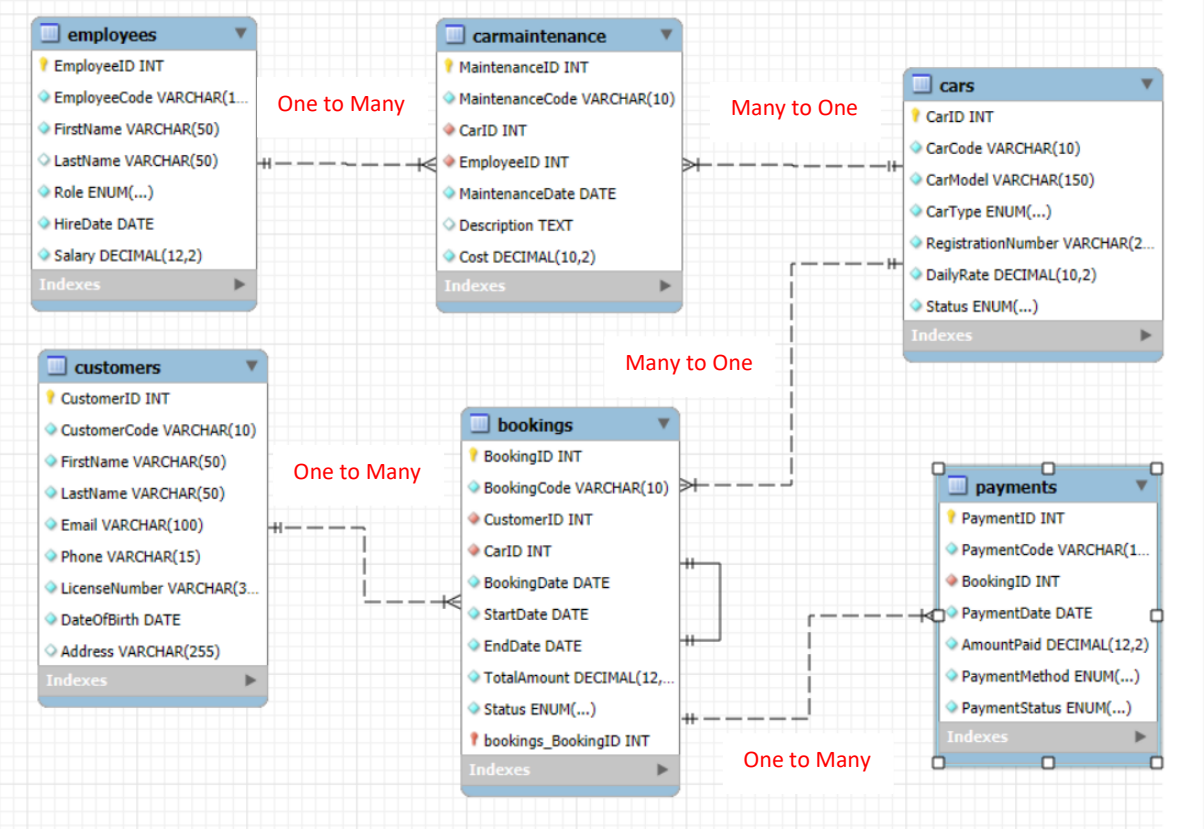
The system enables efficient data storage and retrieval, ensuring timely access to essential information such as car availability, booking history, payment status, and customer details. By maintaining organized and reliable records, it enhances operational efficiency, supports informed decision-making, and improves overall service management.

From a technical perspective, the project incorporates the design of relational tables with primary keys, foreign keys, and integrity constraints to enforce business rules. It further demonstrates the application of SQL through a variety of operations, including data filtering, grouping, aggregation, and ordering. Advanced concepts such as **joins, subqueries, views, window functions, and operators** are employed to derive insights and address complex business queries, such as identifying high-demand vehicles, top customers, and revenue patterns.

In addition, the project highlights the importance of data consistency and accuracy through the implementation of constraints and validations. It demonstrates how a well-designed database can reduce redundancy, ensure referential integrity, and streamline everyday transactions like booking confirmations and payment tracking. By aligning database functionalities with real-world scenarios, the project effectively bridges the gap between theoretical SQL knowledge and practical business applications.

This project not only strengthens proficiency in SQL and database management but also illustrates how structured data handling can optimize business processes in a real-world domain.

ER Diagram:

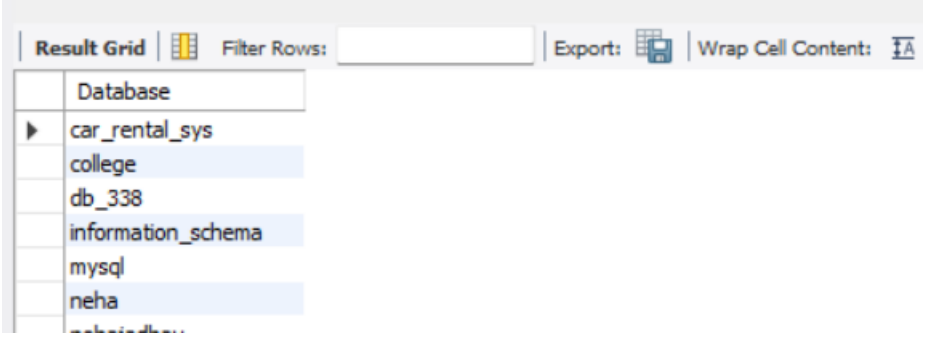

employees
EmployeeID INT
EmployeeCode VARCHAR(1...
FirstName VARCHAR(50)
LastName VARCHAR(50)
Role ENUM(...)
HireDate DATE
Salary DECIMAL(12,2)
Indexes
One to Many
carmaintenance
MaintenanceID INT
MaintenanceCode VARCHAR(10)
CarID INT
EmployeeID INT
MaintenanceDate DATE
Description TEXT
Cost DECIMAL(10,2)
Indexes
Many to One
cars
CarID INT
CarCode VARCHAR(10)
CarModel VARCHAR(150)
CarType ENUM(...)
RegistrationNumber VARCHAR(2...
DailyRate DECIMAL(10,2)
Status ENUM(...)
Indexes
Many to One
customers
CustomerID INT
CustomerCode VARCHAR(10)
FirstName VARCHAR(50)
LastName VARCHAR(50)
Email VARCHAR(100)
Phone VARCHAR(15)
LicenseNumber VARCHAR(3...
DateOfBirth DATE
Address VARCHAR(255)
Indexes
One to Many
bookings
BookingID INT
BookingCode VARCHAR(10)
CustomerID INT
CarID INT
BookingDate DATE
StartDate DATE
EndDate DATE
TotalAmount DECIMAL(12,...
Status ENUM(...)
bookings_BookingID INT
Indexes
payments
PaymentID INT
PaymentCode VARCHAR(1...
BookingID INT
PaymentDate DATE
AmountPaid DECIMAL(12,2)
PaymentMethod ENUM(...)
PaymentStatus ENUM(...)
Indexes
One to Many

Database:

create database car_rental_sys;

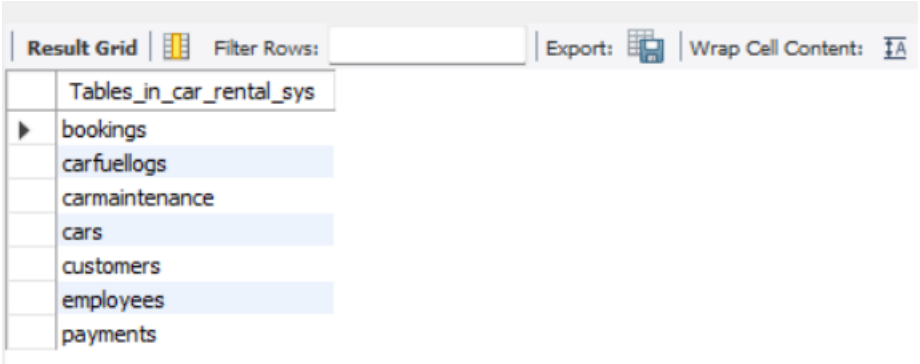
use car_rental_sys;

show databases;



Tables in car_rental_sys Database:

show tables;



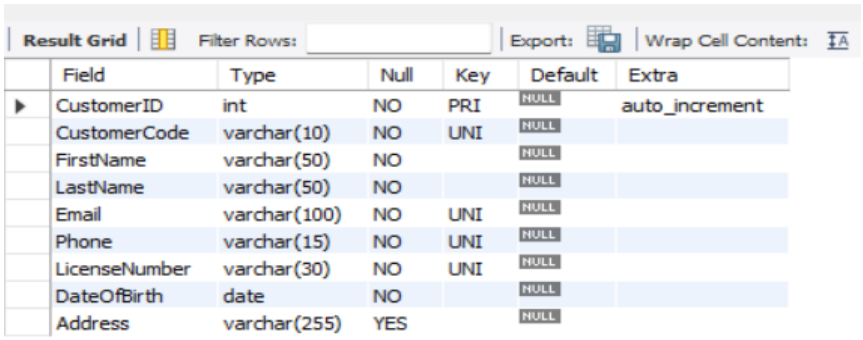
1) DATA DEFINITION LANGUAGE(DDL):

1. Creating Tables:

A) Customers:

CREATE TABLE Customers (CustomerID INT AUTO_INCREMENT PRIMARY KEY, CustomerCode VARCHAR(10) UNIQUE NOT NULL, FirstName VARCHAR(50) NOT NULL, LastName VARCHAR(50) NOT NULL, Email VARCHAR(100) NOT NULL UNIQUE, Phone VARCHAR(15) NOT NULL UNIQUE, LicenseNumber VARCHAR(30) NOT NULL UNIQUE, DateOfBirth DATE NOT NULL, Address VARCHAR(255));

desc Customers;



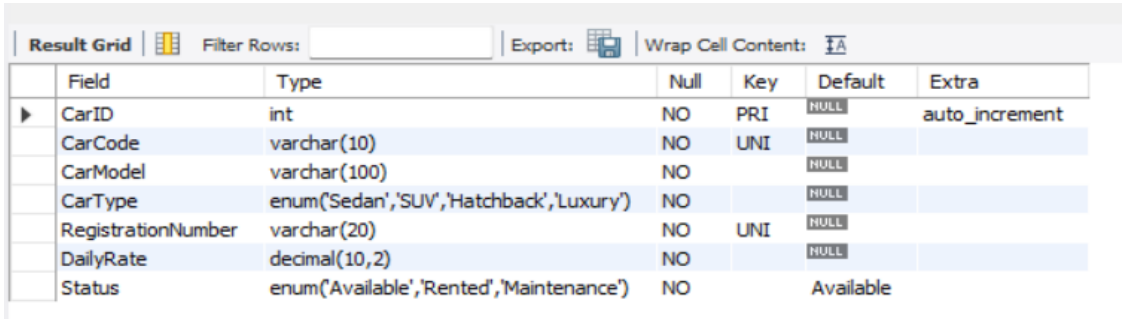
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
CustomerID	int	NO	PRI	NULL	auto_increment
CustomerCode	varchar(10)	NO	UNI	NULL	
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Email	varchar(100)	NO	UNI	NULL	
Phone	varchar(15)	NO	UNI	NULL	
LicenseNumber	varchar(30)	NO	UNI	NULL	
DateOfBirth	date	NO		NULL	
Address	varchar(255)	YES		NULL	

B) Cars:

CREATE TABLE Cars (CarID INT AUTO_INCREMENT PRIMARY KEY, CarCode VARCHAR(10) UNIQUE NOT NULL, CarModel VARCHAR(100) NOT NULL, CarType ENUM('Sedan','SUV','Hatchback','Luxury') NOT NULL, RegistrationNumber VARCHAR(20) NOT NULL UNIQUE, DailyRate DECIMAL(10,2) NOT NULL CHECK (DailyRate > 0), Status ENUM('Available','Rented','Maintenance') NOT NULL DEFAULT 'Available');

desc Cars;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

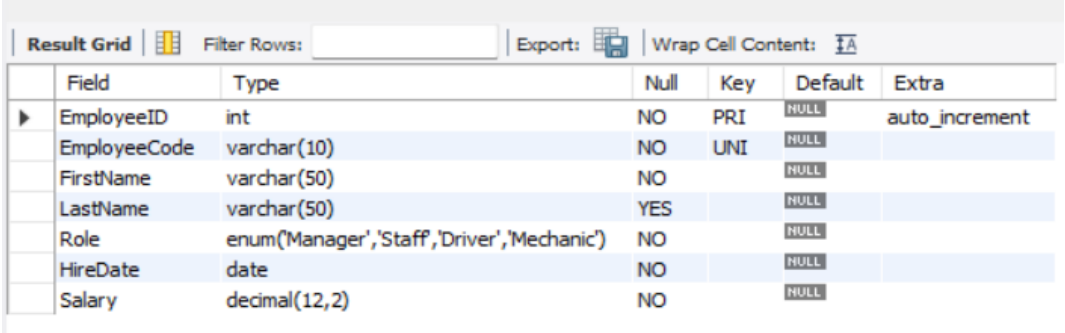
Field	Type	Null	Key	Default	Extra
CarID	int	NO	PRI	NULL	auto_increment
CarCode	varchar(10)	NO	UNI	NULL	
CarModel	varchar(100)	NO		NULL	
CarType	enum('Sedan','SUV','Hatchback','Luxury')	NO		NULL	
RegistrationNumber	varchar(20)	NO	UNI	NULL	
DailyRate	decimal(10,2)	NO		NULL	
Status	enum('Available','Rented','Maintenance')	NO		Available	

C) Employees:

CREATE TABLE Employees (EmployeeID INT AUTO_INCREMENT PRIMARY KEY, EmployeeCode VARCHAR(10) UNIQUE NOT NULL, FirstName VARCHAR(50) NOT NULL, LastName VARCHAR(50), Role

ENUM('Manager','Staff','Driver','Mechanic') NOT NULL, HireDate DATE NOT NULL, Salary DECIMAL(12,2) NOT NULL CHECK (Salary > 5000));

desc Employees;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
EmployeeID	int	NO	PRI	NULL	auto_increment
EmployeeCode	varchar(10)	NO	UNI	NULL	
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	YES		NULL	
Role	enum('Manager','Staff','Driver','Mechanic')	NO		NULL	
HireDate	date	NO		NULL	
Salary	decimal(12,2)	NO		NULL	

D) Bookings:

CREATE TABLE Bookings (BookingID INT AUTO_INCREMENT PRIMARY KEY, BookingCode VARCHAR(10) UNIQUE NOT NULL, CustomerID INT NOT NULL, CarID INT NOT NULL, BookingDate DATE NOT NULL, StartDate DATE NOT NULL, EndDate DATE NOT NULL,

TotalAmount DECIMAL(12,2) NOT NULL CHECK (TotalAmount >= 0),

Status ENUM('Ongoing','Completed','Cancelled') NOT NULL DEFAULT 'Ongoing',

CONSTRAINT fk_booking_customer FOREIGN KEY (CustomerID)

REFERENCES Customers(CustomerID)

ON DELETE CASCADE ON UPDATE CASCADE,

CONSTRAINT fk_booking_car FOREIGN KEY (CarID)

REFERENCES Cars(CarID)

ON DELETE RESTRICT ON UPDATE CASCADE,

CHECK (EndDate > StartDate));

desc Bookings;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
BookingID	int	NO	PRI	NULL	auto_increment
BookingCode	varchar(10)	NO	UNI	NULL	
CustomerID	int	NO	MUL	NULL	
CarID	int	NO	MUL	NULL	
BookingDate	date	NO		NULL	
StartDate	date	NO		NULL	
EndDate	date	NO		NULL	
TotalAmount	decimal(12,2)	NO		NULL	
Status	enum('Ongoing','Completed','Cancelled')	NO		Ongoing	

E) Payments:

```
CREATE TABLE Payments (PaymentID INT AUTO_INCREMENT PRIMARY KEY, BookingID INT NOT NULL, PaymentDate DATE NOT NULL,

AmountPaid DECIMAL(12,2) NOT NULL CHECK (AmountPaid > 0),

PaymentMethod ENUM('Cash','Card','UPI','NetBanking') NOT NULL DEFAULT 'Cash',

PaymentStatus ENUM('Success','Pending','Failed') NOT NULL DEFAULT 'Pending',

CONSTRAINT fk_payment_booking FOREIGN KEY (BookingID)

  REFERENCES Bookings(BookingID)

  ON DELETE CASCADE ON UPDATE CASCADE);

desc Payments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
PaymentID	int	NO	PRI	NULL	auto_increment
BookingID	int	NO	MUL	NULL	
PaymentDate	date	NO		NULL	
AmountPaid	decimal(12,2)	NO		NULL	
PaymentMethod	enum('Cash','Card','UPI','NetBanking')	NO		Cash	
PaymentStatus	enum('Success','Pending','Failed')	NO		Pending	

F) CarMaintenance:

```
CREATE TABLE CarMaintenance (MaintenanceID INT AUTO_INCREMENT PRIMARY KEY, MaintenanceCode VARCHAR(10) UNIQUE NOT NULL, CarID INT NOT NULL, EmployeeID INT NOT NULL, MaintenanceDate DATE NOT NULL, Description TEXT,

Cost DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (Cost >= 0),

CONSTRAINT fk_maintenance_car FOREIGN KEY (CarID)

  REFERENCES Cars(CarID)
```

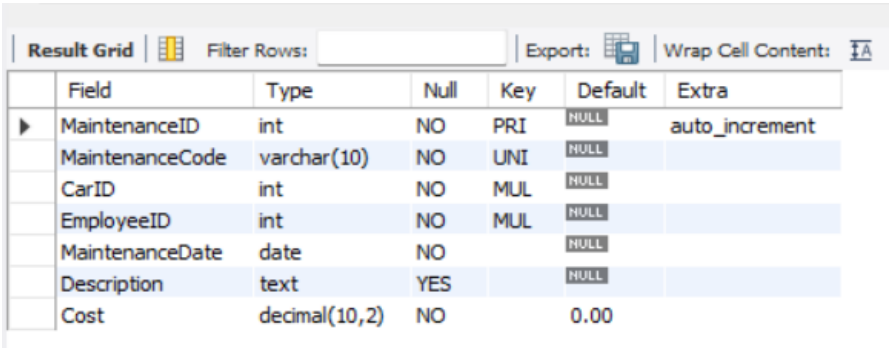
ON DELETE CASCADE ON UPDATE CASCADE,

CONSTRAINT fk_maintenance_employee FOREIGN KEY (EmployeeID)

REFERENCES Employees(EmployeeID)

ON DELETE RESTRICT ON UPDATE CASCADE);

Desc CarMaintenance;



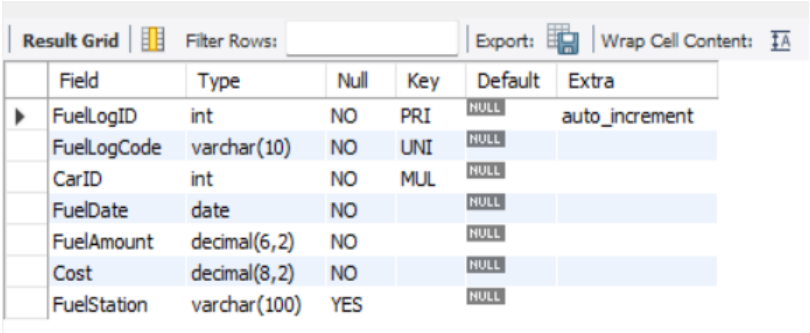
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
MaintenanceID	int	NO	PRI	NULL	auto_increment
MaintenanceCode	varchar(10)	NO	UNI	NULL	
CarID	int	NO	MUL	NULL	
EmployeeID	int	NO	MUL	NULL	
MaintenanceDate	date	NO		NULL	
Description	text	YES		NULL	
Cost	decimal(10,2)	NO		0.00	

G) Car Fual Logs:

CREATE TABLE CarFuelLogs (FuelLogID INT AUTO_INCREMENT PRIMARY KEY, FuelLogCode VARCHAR(10) UNIQUE NOT NULL, CarID INT NOT NULL, FuelDate DATE NOT NULL, FuelAmount DECIMAL(6,2) NOT NULL CHECK (FuelAmount > 0), Cost DECIMAL(8,2) NOT NULL CHECK (Cost >= 0), FuelStation VARCHAR(100), CONSTRAINT fk_fuellog_car FOREIGN KEY (CarID) REFERENCES Cars(CarID) ON DELETE CASCADE ON UPDATE CASCADE);

desc CarFuelLogs;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
FuelLogID	int	NO	PRI	NULL	auto_increment
FuelLogCode	varchar(10)	NO	UNI	NULL	
CarID	int	NO	MUL	NULL	
FuelDate	date	NO		NULL	
FuelAmount	decimal(6,2)	NO		NULL	
Cost	decimal(8,2)	NO		NULL	
FuelStation	varchar(100)	YES		NULL	

2. Alter Tables:

A) Alter Table: Add Column

ALTER TABLE Payments ADD COLUMN PaymentCode VARCHAR(10) UNIQUE NOT NULL AFTER PaymentID;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
PaymentID	int	NO	PRI	NULL	auto_increment
PaymentCode	varchar(10)	NO	UNI	NULL	
BookingID	int	NO	MUL	NULL	
PaymentDate	date	NO		NULL	
AmountPaid	decimal(12,2)	NO		NULL	
PaymentMethod	enum('Cash','Card','UPI','NetBanking')	NO		Cash	
PaymentStatus	enum('Success','Pending','Failed')	NO		Pending	

B) Alter Table: Modify Column

I) Changing size of column:

ALTER TABLE Cars MODIFY COLUMN CarModel VARCHAR(150) NOT NULL;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
CarID	int	NO	PRI	NULL	auto_increment
CarCode	varchar(10)	NO	UNI	NULL	
CarModel	varchar(150)	NO		NULL	
CarType	enum('Sedan','SUV','Hatchback','Luxury')	NO		NULL	
RegistrationNumber	varchar(20)	NO	UNI	NULL	
DailyRate	decimal(10,2)	NO		NULL	
Status	enum('Available','Rented','Maintenance')	NO		Available	

II) Changing datatype of column:

ALTER TABLE CarFuelLogs MODIFY COLUMN FuelDate DATETIME NOT NULL;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
FuelLogID	int	NO	PRI	NULL	auto_increment
FuelLogCode	varchar(10)	NO	UNI	NULL	
CarID	int	NO	MUL	NULL	
FuelDate	datetime	NO		NULL	
FuelAmount	decimal(8,3)	NO		NULL	
Cost	decimal(8,2)	NO		NULL	
FuelStation	varchar(100)	YES		NULL	

C)Alter Table: Rename Column

ALTER TABLE CarFuelLogs RENAME COLUMN FuelDate TO DateOfFuel;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
FuelLogID	int	NO	PRI	NULL	auto_increment
FuelLogCode	varchar(10)	NO	UNI	NULL	
CarID	int	NO	MUL	NULL	
DateOfFuel	datetime	NO		NULL	
FuelAmount	decimal(8,3)	NO		NULL	
Cost	decimal(8,2)	NO		NULL	
FuelStation	varchar(100)	YES		NULL	

D) Alter Table: Rename Table

ALTER TABLE CarFuelLogs RENAME TO FuelLogs;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tables_in_car_rental_sys
bookings
carmaintenance
cars
customers
employees
fuellogs
payments

desc FuelLogs;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
FuelLogID	int	NO	PRI	NULL	auto_increment
FuelLogCode	varchar(10)	NO	UNI	NULL	
CarID	int	NO	MUL	NULL	
DateOfFuel	datetime	NO		NULL	
FuelAmount	decimal(8,3)	NO		NULL	
Cost	decimal(8,2)	NO		NULL	
FuelStation	varchar(100)	YES		NULL	

3. Truncate table:

TRUNCATE TABLE FuelLogs;

SELECT * FROM FuelLogs;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

FuelLogID	FuelLogCode	CarID	DateOfFuel	FuelAmount	Cost	FuelStation
NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. Drop Table:

DROP TABLE FuelLogs;

⊗	82	01:30:22	desc FuelLogs
⊗	83	01:30:27	SELECT * FROM FuelLogs LIMIT 0, 1000

Error Code: 1146. Table 'car_rental_sys.fuellogs' doesn't exist

Error Code: 1146. Table 'car_rental_sys.fuellogs' doesn't exist

2) DATA MANIPULATION LANGUAGE (DML):

1. Insert:

Insert into table: Customers

INSERT INTO Customers (CustomerCode, FirstName, LastName, Email, Phone, LicenseNumber, DateOfBirth, Address)

VALUES

('CUS101', 'Rahul', 'Sharma', 'rahul.sharma@example.com', '9876543210','MH12AB1234','1995-05-12','Pune, Maharashtra'),

……..

('CUS115','Manish','Choudhary','manish.choudhary@example.com','9855566677','MP09CD4455','1992-09-12','Bhopal, Madhya Pradesh');

select * from Customers;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	CustomerCode	FirstName	LastName	Email	Phone	LicenseNumber	DateOfBirth	Address
1	CUS101	Rahul	Sharma	rahul.sharma@example.com	9876543210	MH12AB1234	1995-05-12	Pune, Maharashtra
2	CUS102	Sneha	Patil	sneha.patil@example.com	9823456789	MH14CD5678	1998-09-25	Mumbai, Maharashtra
3	CUS103	Amit	Verma	amit.verma@example.com	9812345678	DL08EF9101	1989-02-10	Delhi
4	CUS104	Priya	Mehta	priya.mehta@example.com	9834567890	GJ01GH1122	1992-11-18	Ahmedabad, Gujarat
5	CUS105	Karan	Joshi	karan.joshi@example.com	9845678901	RJ27IJ3344	1990-07-03	Jaipur, Rajasthan
6	CUS106	Anjali	Kapoor	anjali.kapoor@example.com	9856789012	KA05KL5566	1996-12-22	Bengaluru, Karnataka
7	CUS107	Vikram	Singh	vikram.singh@example.com	9867890123	UP16MN7788	1988-04-14	Lucknow, Uttar Pradesh
8	CUS108	Neha	Rao	neha.rao@example.com	9878901234	TS09OP9900	1997-08-05	Hyderabad, Telangana
9	CUS109	Rohit	Nair	rohit.nair@example.com	9889012345	KL07QR2233	1991-01-27	Kochi, Kerala
10	CUS110	Meera	Desai	meera.desai@example.com	9890123456	MH31ST4455	1994-10-16	Nagpur, Maharashtra
11	CUS111	Arjun	Kulkarni	arjun.kulkarni@example.com	9811122233	MH20UV6677	1993-03-09	Aurangabad, Maharas…
12	CUS112	Simran	Kaur	simran.kaur@example.com	9822233344	PB10WX8899	1995-07-19	Chandigarh, Punjab
13	CUS113	Dev	Malhotra	dev.malhotra@example.com	9833344455	HR26YZ0011	1987-12-30	Gurgaon, Haryana
14	CUS114	Ishita	Roy	ishita.roy@example.com	9844455566	WB20AB2233	1999-06-21	Kolkata, West Bengal
15	CUS115	Manish	Choudhary	manish.choudhary@exampl…	9855566677	MP09CD4455	1992-09-12	Bhopal, Madhya Pradesh
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. Update

Example: Update Status column from Cars table where CarID is 8

UPDATE Cars SET Status = 'Available' WHERE CarID = 8;

Select * from Cars;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
1	CAR101	Swift Dzire	Sedan	MH12AB1234	1200.00	Available
2	CAR102	i20	Hatchback	MH14CD5678	1500.00	Rented
3	CAR103	City	Sedan	DL08EF9101	2000.00	Available
4	CAR104	EcoSport	SUV	GJ01GH1122	1800.00	Available
5	CAR105	Baleno	Hatchback	RJ27IJ3344	1700.00	Available
6	CAR106	Creta	SUV	KA05KL5566	2200.00	Maintenance
7	CAR107	Innova Crysta	SUV	UP16MN7788	3000.00	Available
8	CAR108	Altroz	Hatchback	TS09OP9900	1600.00	Available
9	CAR109	Fortuner	Luxury	KL07QR2233	4500.00	Maintenance
10	CAR110	XUV500	SUV	MH31ST4455	2800.00	Available
NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. Delete

Example: Delete one record from Payments table where PaymentID is 10

DELETE from Payments where PaymentID=10;

Select * from Payments;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

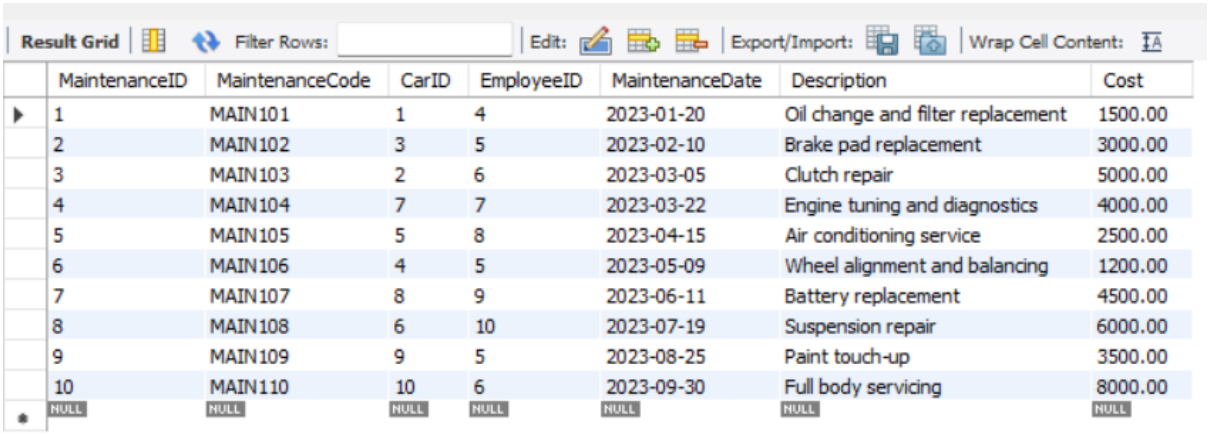
PaymentID	PaymentCode	BookingID	PaymentDate	AmountPaid	PaymentMethod	PaymentStatus
1	PAY101	1	2023-01-15	3600.00	Card	Success
2	PAY102	2	2023-02-08	4000.00	UPI	Success
3	PAY103	3	2023-03-04	4500.00	Cash	Failed
4	PAY104	4	2023-04-15	3400.00	NetBanking	Success
5	PAY105	5	2023-05-23	3600.00	Card	Success
6	PAY106	6	2023-06-21	6000.00	UPI	Pending
7	PAY107	7	2023-07-29	6600.00	Cash	Success
8	PAY108	8	2023-08-13	3200.00	Card	Failed
9	PAY109	9	2023-09-05	13500.00	NetBanking	Success
NULL	NULL	NULL	NULL	NULL	NULL	NULL

3) DATA QUERY LANGUAGE (DQL):

1. Select

A) Selecting query for entire data

select * from CarMaintenance;

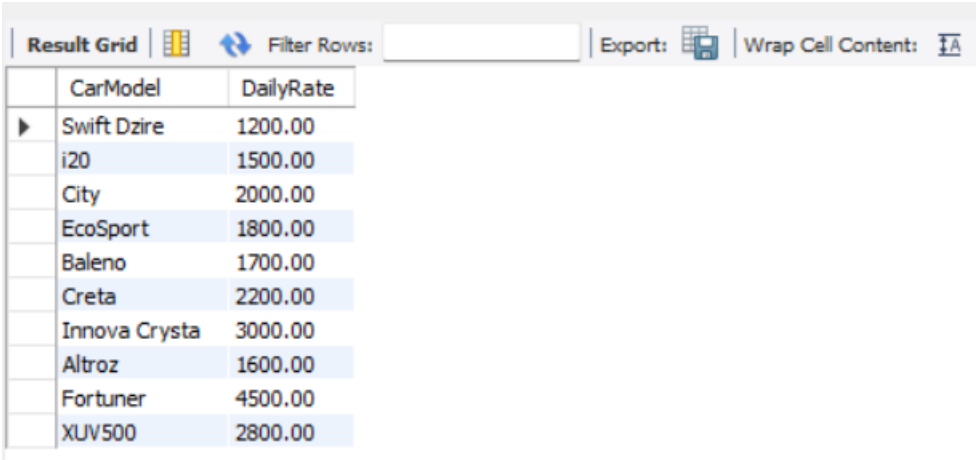


MaintenanceID	MaintenanceCode	CarID	EmployeeID	MaintenanceDate	Description	Cost
1	MAIN101	1	4	2023-01-20	Oil change and filter replacement	1500.00
2	MAIN102	3	5	2023-02-10	Brake pad replacement	3000.00
3	MAIN103	2	6	2023-03-05	Clutch repair	5000.00
4	MAIN104	7	7	2023-03-22	Engine tuning and diagnostics	4000.00
5	MAIN105	5	8	2023-04-15	Air conditioning service	2500.00
6	MAIN106	4	5	2023-05-09	Wheel alignment and balancing	1200.00
7	MAIN107	8	9	2023-06-11	Battery replacement	4500.00
8	MAIN108	6	10	2023-07-19	Suspension repair	6000.00
9	MAIN109	9	5	2023-08-25	Paint touch-up	3500.00
10	MAIN110	10	6	2023-09-30	Full body servicing	8000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

B) Selecting specific data

Example: Select CarModel and Dailyrate from Cars Table

select CarModel, DailyRate from Cars;



CarModel	DailyRate
Swift Dzire	1200.00
i20	1500.00
City	2000.00
EcoSport	1800.00
Baleno	1700.00
Creta	2200.00
Innova Crysta	3000.00
Altroz	1600.00
Fortuner	4500.00
XUV500	2800.00

2. Alisement (as):

Example: Select query with changing column name

select Salary as Monthly_Salary from Employees;

Result Grid | Filte

Monthly_Salary
55000.00
28000.00
18000.00
22000.00
30000.00
17000.00
25000.00
27000.00
60000.00
16000.00

3. DISTINCT QUERY:

Example: Display distinct Car Types

SELECT DISTINCT CarType FROM Cars;

Result Grid | Filter R

CarType
Sedan
Hatchback
SUV
Luxury

CLAUSES

A) Where Clause:

Example 1: Retrieve information about BookingCode and TotalAmount where booking Status is 'Ongoing' from Bookings table:

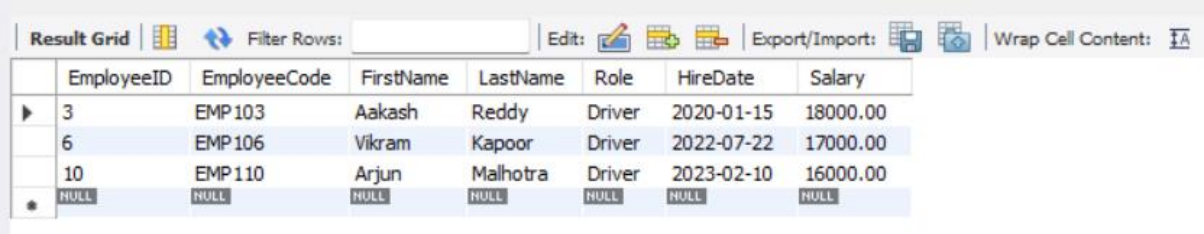
select BookingCode, TotalAmount from Bookings where Status='Ongoing';

Result Grid | Filter Rows:

BookingCode	TotalAmount
BKG106	12000.00
BKG110	5600.00

Example 2: Retrieve information about employees where Role is 'Driver' from Employees table

select * from Employees where Role='Driver';



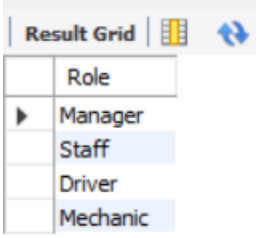
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

EmployeeID	EmployeeCode	FirstName	LastName	Role	HireDate	Salary
3	EMP103	Aakash	Reddy	Driver	2020-01-15	18000.00
6	EMP106	Vikram	Kapoor	Driver	2022-07-22	17000.00
10	EMP110	Arjun	Malhotra	Driver	2023-02-10	16000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

B) Group by Clause:

Example 1: List all roles in employees table

SELECT Role FROM Employees GROUP BY Role;



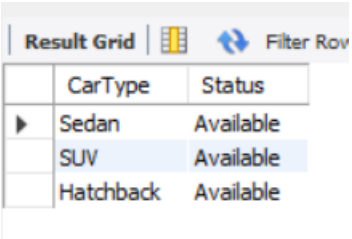
Result Grid

Role
Manager
Staff
Driver
Mechanic

Where + Group by

Example 2: Group cars by their type, but only those that are "Available" from Cars table

SELECT CarType, Status FROM Cars WHERE Status = 'Available' GROUP BY CarType, Status;



Result Grid | Filter Rov

CarType	Status
Sedan	Available
SUV	Available
Hatchback	Available

C) Having Clause

Example 1: Show car data groups where the status is "Available" only from Cars table

SELECT Status, COUNT(*) AS TotalCars FROM Cars GROUP BY Status HAVING Status = 'Available';

Result Grid | Filter Rows:

Status	TotalCars
Available	7

where + group by + having

Example 2: Show only bookings made after 2023-01-01, and then filter groups where the booking status is "Completed" from Bookings table

SELECT Status, COUNT(*) AS TotalBookings FROM Bookings WHERE BookingDate > '2023-01-01' GROUP BY Status HAVING Status = 'Completed';

Result Grid | Filter Rows:

Status	TotalBookings
Completed	6

D) Order By Clause:

Example 1: Show employees ordered by salary (descending) from Employee table

SELECT EmployeeCode, FirstName, LastName, Role, Salary FROM Employees ORDER BY Salary DESC;

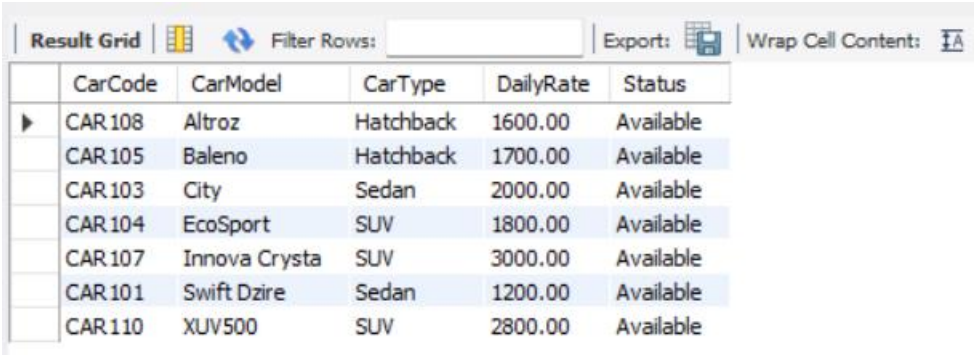
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeCode	FirstName	LastName	Role	Salary
EMP109	Meena	Chopra	Manager	60000.00
EMP101	Rajesh	Sharma	Manager	55000.00
EMP105	Kiran	Patil	Staff	30000.00
EMP102	Suman	Verma	Staff	28000.00
EMP108	Suresh	Menon	Staff	27000.00
EMP107	Anita	Desai	Mechanic	25000.00
EMP104	Priya	Iyer	Mechanic	22000.00
EMP103	Aakash	Reddy	Driver	18000.00
EMP106	Vikram	Kapoor	Driver	17000.00
EMP110	Arjun	Malhotra	Driver	16000.00

Where + Order by Clause:

Example 2: List only available cars, ordered by model name (ascending) from Cars table

SELECT CarCode, CarModel, CarType, DailyRate, Status FROM Cars WHERE Status = 'Available' ORDER BY CarModel;



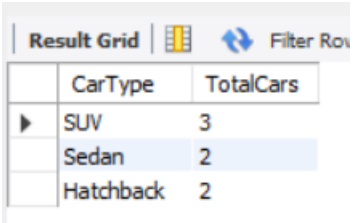
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CarCode	CarModel	CarType	DailyRate	Status
CAR108	Altroz	Hatchback	1600.00	Available
CAR105	Baleno	Hatchback	1700.00	Available
CAR103	City	Sedan	2000.00	Available
CAR104	EcoSport	SUV	1800.00	Available
CAR107	Innova Crysta	SUV	3000.00	Available
CAR101	Swift Dzire	Sedan	1200.00	Available
CAR110	XUV500	SUV	2800.00	Available

Where + Group by + Order by

Example 3: Get cars grouped by type (only available ones), ordered by total count descending from Cars table

SELECT CarType, COUNT(*) AS TotalCars FROM Cars WHERE Status = 'Available' GROUP BY CarType ORDER BY TotalCars DESC;



Result Grid | Filter Rou

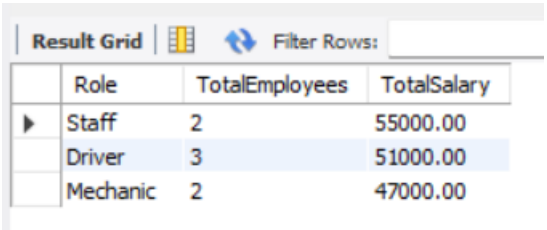
CarType	TotalCars
SUV	3
Sedan	2
Hatchback	2

Where + Group by + Having + Order by

Example 4: Get Employees grouped by role, only those hired after 2019, keeping only roles with more than 1 employee, ordered by salary sum descending

SELECT Role, COUNT(*) AS TotalEmployees, SUM(Salary) AS TotalSalary FROM Employees

WHERE HireDate >= '2019-01-01' GROUP BY Role HAVING COUNT(*) > 1 ORDER BY TotalSalary DESC;



Result Grid | Filter Rows:

Role	TotalEmployees	TotalSalary
Staff	2	55000.00
Driver	3	51000.00
Mechanic	2	47000.00

E) Limit Clause

Example 1: Show just the first 5 customers

SELECT * FROM Customers LIMIT 5;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

CustomerID	CustomerCode	FirstName	LastName	Email	Phone	LicenseNumber	DateOfBirth	Address
1	CUS101	Rahul	Sharma	rahul.sharma@example.com	9876543210	MH12AB1234	1995-05-12	Pune, Maharashtra
2	CUS102	Sneha	Patil	sneha.patil@example.com	9823456789	MH14CD5678	1998-09-25	Mumbai, Maharashtra
3	CUS103	Amit	Verma	amit.verma@example.com	9812345678	DL08EF9101	1989-02-10	Delhi
4	CUS104	Priya	Mehta	priya.mehta@example.com	9834567890	GJ01GH1122	1992-11-18	Ahmedabad, Gujarat
5	CUS105	Karan	Joshi	karan.joshi@example.com	9845678901	RJ27IJ3344	1990-07-03	Jaipur, Rajasthan
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

LIMIT + WHERE

Example 2: Show 3 available cars only

SELECT CarCode, CarModel, Status FROM Cars WHERE Status = 'Available' LIMIT 3;

Result Grid | Filter Rows:

CarCode	CarModel	Status
CAR101	Swift Dzire	Available
CAR103	City	Available
CAR104	EcoSport	Available

LIMIT + WHERE + GROUP BY

Example 3: Show 2 car statuses with their counts, but only for cars where DailyRate is greater than equal to 1800

SELECT Status, COUNT(*) AS TotalCars FROM Cars WHERE DailyRate >= 1800 GROUP BY Status LIMIT 2;

Result Grid | Filter Rows:

Status	TotalCars
Available	4
Maintenance	2

LIMIT + WHERE + GROUP BY + HAVING

Example 4: Show up to 2 roles that have more than 1 employee, hired after 2019

SELECT Role, COUNT(*) AS TotalEmployees FROM Employees WHERE HireDate >= '2019-01-01' GROUP BY Role HAVING COUNT(*) > 1 LIMIT 2;

Result Grid | Filter Rows:

Role	TotalEmployees
Staff	2
Driver	3

LIMIT + WHERE + GROUP BY + HAVING + ORDER BY

Example 5: Show top 3 customer IDs who made bookings after 2021, only if they booked more than equal to 1, ordered by total bookings

SELECT CustomerID, COUNT(*) AS TotalBookings FROM Bookings WHERE StartDate >= '2021-01-01' GROUP BY CustomerID HAVING COUNT(*) >= 1 ORDER BY TotalBookings DESC LIMIT 3;

Result Grid | Filter Rows:

CustomerID	TotalBookings
1	1
2	1
3	1

OPERATORS

A] ARITHMATIC OPERATORS

1) Addition(+) and Multiplication(*)

Example: Calculate total amount plus tax (assuming 18% GST)

SELECT BookingCode, TotalAmount, TotalAmount + TotalAmount * 0.18 AS AmountWithGST FROM Bookings;

Result Grid | Filter Rows:

BookingCode	TotalAmount	AmountWithGST
BKG101	3600.00	4248.0000
BKG102	4000.00	4720.0000
BKG103	4500.00	5310.0000
BKG104	3400.00	4012.0000
BKG105	3600.00	4248.0000
BKG106	12000.00	14160.0000
BKG107	6600.00	7788.0000
BKG108	3200.00	3776.0000
BKG109	13500.00	15930.0000
BKG110	5600.00	6608.0000

2) Subtraction (-)

Example: Calculate remaining payment if a customer has paid partially

SELECT p.PaymentCode, b.BookingCode, b.TotalAmount - p.AmountPaid AS RemainingAmount FROM Payments p JOIN Bookings b ON p.BookingID = b.BookingID;

Result Grid | Filter Rows:

PaymentCode	BookingCode	RemainingAmount
PAY101	BKG101	0.00
PAY102	BKG102	0.00
PAY103	BKG103	0.00
PAY104	BKG104	0.00
PAY105	BKG105	0.00
PAY106	BKG106	6000.00
PAY107	BKG107	0.00
PAY108	BKG108	0.00
PAY109	BKG109	0.00

3) Division (/)

Example: Find average daily payment (if TotalAmount is for multiple days)

SELECT b.BookingCode, c.CarModel, b.TotalAmount / DATEDIFF(b.EndDate, b.StartDate) AS AvgDailyPayment FROM Bookings b JOIN Cars c ON b.CarID = c.CarID;

Result Grid | Filter Rows:

BookingCode	CarModel	AvgDailyPayment
BKG101	Swift Dzire	1200.000000
BKG103	i20	1500.000000
BKG102	City	2000.000000
BKG105	EcoSport	1800.000000
BKG104	Baleno	1700.000000
BKG107	Creta	2200.000000
BKG106	Innova Crysta	3000.000000
BKG108	Altroz	1600.000000
BKG109	Fortuner	4500.000000
BKG110	XUV500	2800.000000

4) Modulus (%)

Example: Check which bookings fall on odd or even booking ID

SELECT BookingCode, BookingID, CASE WHEN BookingID % 2 = 0 THEN 'Even' ELSE 'Odd' END AS BookingType FROM Bookings;

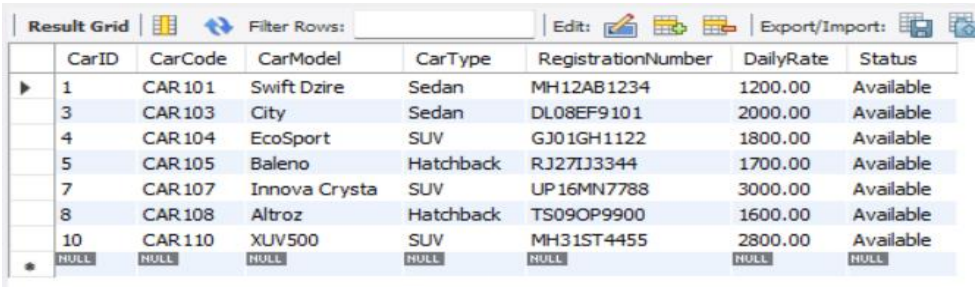
Result Grid | Filter Rows:

BookingCode	BookingID	BookingType
BKG101	1	Odd
BKG102	2	Even
BKG103	3	Odd
BKG104	4	Even
BKG105	5	Odd
BKG106	6	Even
BKG107	7	Odd
BKG108	8	Even
BKG109	9	Odd
BKG110	10	Even

B] COMPARISON OPERATORS

1) Equal

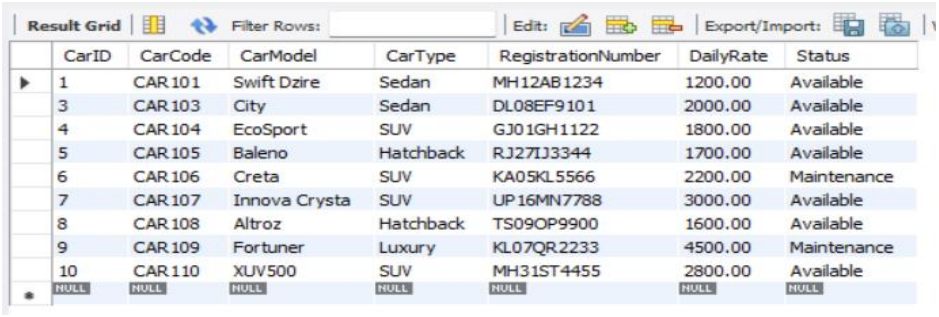
SELECT * FROM Cars WHERE Status = 'Available';



CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
1	CAR101	Swift Dzire	Sedan	MH12AB1234	1200.00	Available
3	CAR103	City	Sedan	DL08EF9101	2000.00	Available
4	CAR104	EcoSport	SUV	GJ01GH1122	1800.00	Available
5	CAR105	Baleno	Hatchback	RJ27IJ3344	1700.00	Available
7	CAR107	Innova Crysta	SUV	UP16MN7788	3000.00	Available
8	CAR108	Altroz	Hatchback	TS09OP9900	1600.00	Available
10	CAR110	XUV500	SUV	MH31ST4455	2800.00	Available

2) Not equal

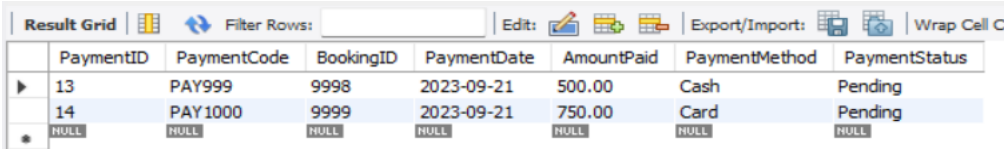
SELECT * FROM Cars WHERE Status <> 'Rented';



CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
1	CAR101	Swift Dzire	Sedan	MH12AB1234	1200.00	Available
3	CAR103	City	Sedan	DL08EF9101	2000.00	Available
4	CAR104	EcoSport	SUV	GJ01GH1122	1800.00	Available
5	CAR105	Baleno	Hatchback	RJ27IJ3344	1700.00	Available
6	CAR106	Creta	SUV	KA05KL5566	2200.00	Maintenance
7	CAR107	Innova Crysta	SUV	UP16MN7788	3000.00	Available
8	CAR108	Altroz	Hatchback	TS09OP9900	1600.00	Available
9	CAR109	Fortuner	Luxury	KL07QR2233	4500.00	Maintenance
10	CAR110	XUV500	SUV	MH31ST4455	2800.00	Available

3) Less than or equal

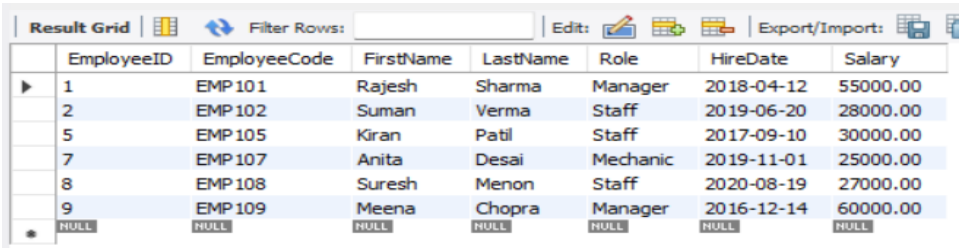
SELECT * FROM Payments WHERE AmountPaid <= 1500;



PaymentID	PaymentCode	BookingID	PaymentDate	AmountPaid	PaymentMethod	PaymentStatus
13	PAY999	9998	2023-09-21	500.00	Cash	Pending
14	PAY1000	9999	2023-09-21	750.00	Card	Pending

4) Greater than or equal

SELECT * FROM Employees WHERE Salary >= 25000;

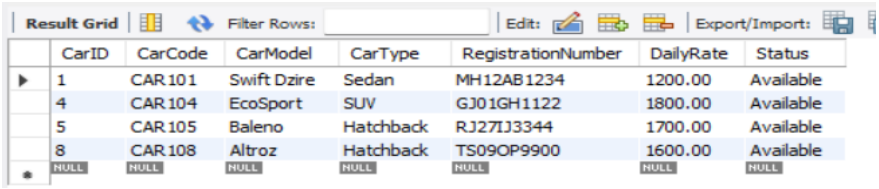


EmployeeID	EmployeeCode	FirstName	LastName	Role	HireDate	Salary
1	EMP101	Rajesh	Sharma	Manager	2018-04-12	55000.00
2	EMP102	Suman	Verma	Staff	2019-06-20	28000.00
5	EMP105	Kiran	Patil	Staff	2017-09-10	30000.00
7	EMP107	Anita	Desai	Mechanic	2019-11-01	25000.00
8	EMP108	Suresh	Menon	Staff	2020-08-19	27000.00
9	EMP109	Meena	Chopra	Manager	2016-12-14	60000.00

C] LOGICAL OPERATORS

1) AND

SELECT * FROM Cars WHERE Status = 'Available' AND DailyRate < 2000;



CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
1	CAR101	Swift Dzire	Sedan	MH12AB1234	1200.00	Available
4	CAR104	EcoSport	SUV	GJ01GH1122	1800.00	Available
5	CAR105	Baleno	Hatchback	RJ27IJ3344	1700.00	Available
8	CAR108	Altroz	Hatchback	TS09OP9900	1600.00	Available

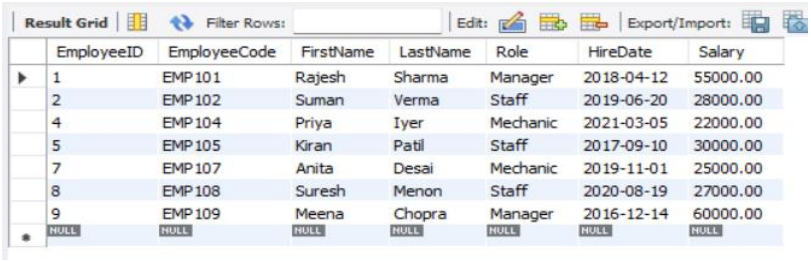
2) OR

SELECT * FROM Cars WHERE Status = 'Unavailable' OR Status = 'Maintenance';

CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
6	CAR106	Creta	SUV	KA05KL5566	2200.00	Maintenance
9	CAR109	Fortuner	Luxury	KL07QR2233	4500.00	Maintenance

3) NOT

SELECT * FROM Employees WHERE NOT Role = 'Driver';

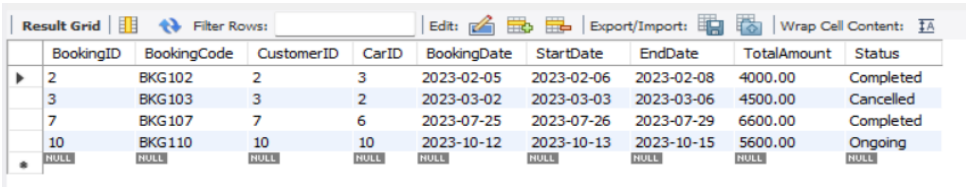


EmployeeID	EmployeeCode	FirstName	LastName	Role	HireDate	Salary
1	EMP101	Rajesh	Sharma	Manager	2018-04-12	55000.00
2	EMP102	Suman	Verma	Staff	2019-06-20	28000.00
4	EMP104	Priya	Iyer	Mechanic	2021-03-05	22000.00
5	EMP105	Kiran	Patil	Staff	2017-09-10	30000.00
7	EMP107	Anita	Desai	Mechanic	2019-11-01	25000.00
8	EMP108	Suresh	Menon	Staff	2020-08-19	27000.00
9	EMP109	Meena	Chopra	Manager	2016-12-14	60000.00

D] SPECIAL OPERATORS

1) BETWEEN

SELECT * FROM Bookings WHERE TotalAmount BETWEEN 1000 AND 3000;

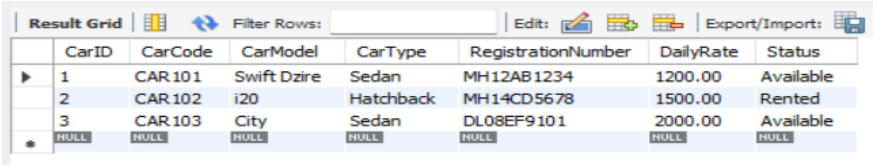


BookingID	BookingCode	CustomerID	CarID	BookingDate	StartDate	EndDate	TotalAmount	Status
2	BKG102	2	3	2023-02-05	2023-02-06	2023-02-08	4000.00	Completed
3	BKG103	3	2	2023-03-02	2023-03-03	2023-03-06	4500.00	Cancelled
7	BKG107	7	6	2023-07-25	2023-07-26	2023-07-29	6600.00	Completed
10	BKG110	10	10	2023-10-12	2023-10-13	2023-10-15	5600.00	Ongoing

2) IN / NOT IN

Example 1: IN

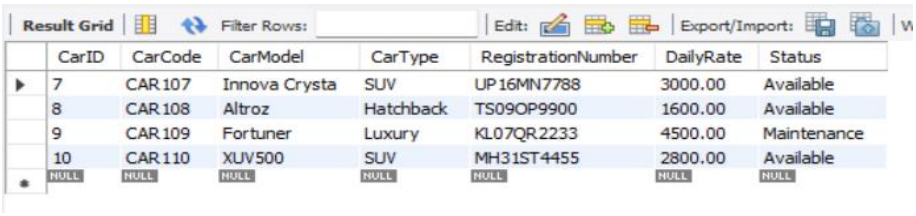
SELECT * FROM Cars WHERE CarID IN (101, 102, 103);



CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
1	CAR101	Swift Dzire	Sedan	MH12AB1234	1200.00	Available
2	CAR102	i20	Hatchback	MH14CD5678	1500.00	Rented
3	CAR103	City	Sedan	DL08EF9101	2000.00	Available

Example 2: IN NOT

SELECT * FROM Cars WHERE CarID NOT IN (1, 2, 3, 4, 5, 6);

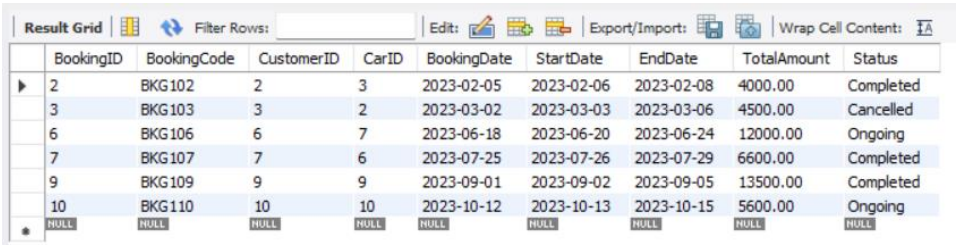


CarID	CarCode	CarModel	CarType	RegistrationNumber	DailyRate	Status
7	CAR107	Innova Crysta	SUV	UP16MN7788	3000.00	Available
8	CAR108	Altroz	Hatchback	TS09OP9900	1600.00	Available
9	CAR109	Fortuner	Luxury	KL07QR2233	4500.00	Maintenance
10	CAR110	XUV500	SUV	MH31ST4455	2800.00	Available

3) ANY (with subquery)

Example: Find bookings with amount greater than any booking made by CustomerID = 1

SELECT * FROM Bookings WHERE TotalAmount > ANY (SELECT TotalAmount FROM Bookings WHERE CustomerID = 1);

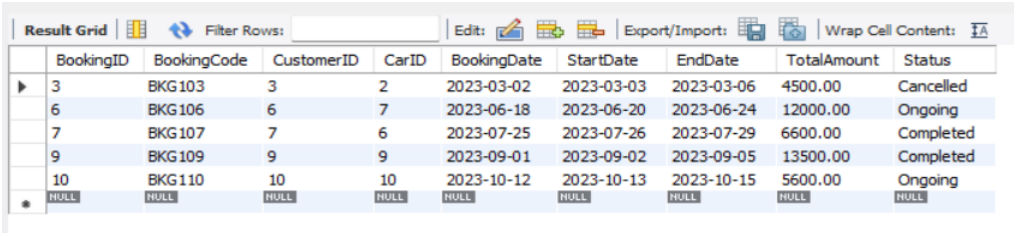


BookingID	BookingCode	CustomerID	CarID	BookingDate	StartDate	EndDate	TotalAmount	Status
2	BKG102	2	3	2023-02-05	2023-02-06	2023-02-08	4000.00	Completed
3	BKG103	3	2	2023-03-02	2023-03-03	2023-03-06	4500.00	Cancelled
6	BKG106	6	7	2023-06-18	2023-06-20	2023-06-24	12000.00	Ongoing
7	BKG107	7	6	2023-07-25	2023-07-26	2023-07-29	6600.00	Completed
9	BKG109	9	9	2023-09-01	2023-09-02	2023-09-05	13500.00	Completed
10	BKG110	10	10	2023-10-12	2023-10-13	2023-10-15	5600.00	Ongoing

4) ALL (with subquery)

Example: Find bookings with amount greater than all bookings of CustomerID = 2

SELECT * FROM Bookings WHERE TotalAmount > ALL (SELECT TotalAmount FROM Bookings WHERE CustomerID = 2);



BookingID	BookingCode	CustomerID	CarID	BookingDate	StartDate	EndDate	TotalAmount	Status
3	BKG103	3	2	2023-03-02	2023-03-03	2023-03-06	4500.00	Cancelled
6	BKG106	6	7	2023-06-18	2023-06-20	2023-06-24	12000.00	Ongoing
7	BKG107	7	6	2023-07-25	2023-07-26	2023-07-29	6600.00	Completed
9	BKG109	9	9	2023-09-01	2023-09-02	2023-09-05	13500.00	Completed
10	BKG110	10	10	2023-10-12	2023-10-13	2023-10-15	5600.00	Ongoing

5) IS NULL / IS NOT NULL

Example 1: IS NULL

SELECT * FROM Customers WHERE Address IS NULL;

CustomerID	Address
NULL	NULL

Example 2: IS NOT NULL

SELECT * FROM Customers WHERE Address IS NOT NULL;

Result Grid | Filter Rows:

CustomerID	Address
1	Pune, Maharashtra
2	Mumbai, Maharashtra
3	Delhi
4	Ahmedabad, Gujarat
5	Jaipur, Rajasthan
6	Bengaluru, Karnataka
7	Lucknow, Uttar Pradesh
8	Hyderabad, Telangana
9	Kochi, Kerala
10	Nagpur, Maharashtra
11	Aurangabad, Maharas...
12	Chandigarh, Punjab
13	Gurgaon, Haryana
14	Kolkata, West Bengal
15	Bhopal, Madhya Pradesh
NULL	NULL

6) LIKE / NOT LIKE

Example 1: LIKE: Names starting with 'R'

SELECT * FROM Customers WHERE FirstName LIKE 'R%';

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	CustomerCode	FirstName	LastName	Email	Phone	LicenseNumber	DateOfBirth	Address
1	CUS101	Rahul	Sharma	rahul.sharma@example.com	9876543210	MH12AB1234	1995-05-12	Pune, Maharashtra
9	CUS109	Rohit	Nair	rohit.nair@example.com	9889012345	KL07QR2233	1991-01-27	Kochi, Kerala
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Example 2: NOT LIKE: Emails NOT containing 'example'

SELECT * FROM Customers WHERE Email NOT LIKE '%example%';

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	CustomerCode	FirstName	LastName	Email	Phone	LicenseNumber	DateOfBirth	Address
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

7) UNION

Example: Combine cars from two types

SELECT CarID, CarModel, CarType FROM Cars WHERE CarType = 'Sedan'

UNION

SELECT CarID, CarModel, CarType FROM Cars WHERE CarType = 'SUV';

Result Grid | Filter Rows:

CarID	CarModel	CarType
1	Swift Dzire	Sedan
3	City	Sedan
4	EcoSport	SUV
6	Creta	SUV
7	Innova Crysta	SUV
10	XUV500	SUV

E] AGGREGATE FUNCTIONS

1) COUNT()

Example: Count total bookings per car:

SELECT CarID, COUNT(*) AS TotalBookings FROM Bookings GROUP BY CarID;

Result Grid | Filter Rov

CarID	TotalBookings
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1

2) SUM()

Example: Total amount received:

SELECT SUM(AmountPaid) AS TotalReceived FROM Payments;

Result Grid

TotalReceived
49650.00

3) AVG()

Example: Average daily rate of cars:

SELECT AVG(DailyRate) AS AvgDailyRate FROM Cars;

Result Grid

AvgDailyRate
2230.000000

4) MIN()

Example: Lowest car rental cost:

SELECT MIN(DailyRate) AS MinDailyRate FROM Cars;

Result Grid

MinDailyRate
1200.00

5) MAX()

Example: Highest salary among employees:

SELECT MAX(Salary) AS MaxSalary FROM Employees;

Result Grid

	MaxSalary
▶	60000.00

SUBQUERIES

1) Single-Row Subqueries:

Example 1: Find the customer with the earliest booking date

```
SELECT FirstName, LastName FROM Customers
WHERE CustomerID = (SELECT CustomerID FROM Bookings ORDER BY BookingDate LIMIT 1);
```

Result Grid Filter Rows:

	FirstName	LastName
▶	Rahul	Sharma

Example 2: Get the car with the highest daily rate

```
SELECT CarCode, CarModel, DailyRate FROM Cars
WHERE DailyRate = (SELECT MAX(DailyRate) FROM Cars);
```

Result Grid Filter Rows:

	CarCode	CarModel	DailyRate
▶	CAR109	Fortuner	4500.00

2) Multi-Row Subqueries:

Example 1: Get employees who are mechanics or drivers

```
SELECT EmployeeCode, FirstName, LastName, Role FROM Employees
WHERE Role = ANY (SELECT Role FROM Employees WHERE Role IN ('Mechanic','Driver'));
```

Result Grid | Filter Rows: | Export

EmployeeCode	FirstName	LastName	Role
EMP103	Aakash	Reddy	Driver
EMP104	Priya	Iyer	Mechanic
EMP106	Vikram	Kapoor	Driver
EMP107	Anita	Desai	Mechanic
EMP110	Arjun	Malhotra	Driver

Example 2: Get bookings where the total amount is higher than some bookings for the same car type

```
SELECT BookingCode, CustomerID, TotalAmount, CarID FROM Bookings b1

WHERE TotalAmount > ANY (SELECT TotalAmount FROM Bookings b2 JOIN Cars c ON b2.CarID =
c.CarID WHERE c.CarType = 'SUV');
```

Result Grid | Filter Rows: | Export:

BookingCode	CustomerID	TotalAmount	CarID
BKG102	2	4000.00	3
BKG103	3	4500.00	2
BKG106	6	12000.00	7
BKG107	7	6600.00	6
BKG109	9	13500.00	9
BKG110	10	5600.00	10

3) Multi-Column Subqueries:

Example 1: Find bookings that match specific car and customer pairs from completed bookings

```
SELECT BookingCode, CustomerID, CarID FROM Bookings

WHERE (CustomerID, CarID) IN (SELECT CustomerID, CarID FROM Bookings WHERE Status =
'Completed');
```

Result Grid | Filter Rows:

BookingCode	CustomerID	CarID
BKG101	1	1
BKG102	2	3
BKG104	4	5
BKG105	5	4
BKG107	7	6
BKG109	9	9

Example 2: Find cars and their daily rates that match exactly the lowest daily rate per car type

```
SELECT CarCode, CarType, DailyRate FROM Cars

WHERE (CarType, DailyRate) IN (SELECT CarType, MIN(DailyRate) FROM Cars GROUP BY CarType);
```

Result Grid | Filter Rows:

CarCode	CarType	DailyRate
CAR101	Sedan	1200.00
CAR102	Hatchback	1500.00
CAR104	SUV	1800.00
CAR109	Luxury	4500.00

4) Multi-Table Subqueries:

Example 1: Customers who booked cars with daily rate higher than the average daily rate of all SUVs

```
SELECT CustomerID, BookingCode, TotalAmount, CarID FROM Bookings b

WHERE TotalAmount > (SELECT AVG(DailyRate) FROM Cars c JOIN Bookings b2 ON c.CarID = b2.CarID
WHERE c.CarType = 'SUV');
```

Result Grid | Filter Rows: | Expo

CustomerID	BookingCode	TotalAmount	CarID
1	BKG101	3600.00	1
2	BKG102	4000.00	3
3	BKG103	4500.00	2
4	BKG104	3400.00	5
5	BKG105	3600.00	4
6	BKG106	12000.00	7
7	BKG107	6600.00	6
8	BKG108	3200.00	8
9	BKG109	13500.00	9
10	BKG110	5600.00	10

Example 2: Find cars whose total maintenance cost exceeds the average maintenance cost for cars of the same type

```
SELECT CarID, MaintenanceCode, Cost FROM CarMaintenance cm

WHERE Cost > (SELECT AVG(Cost) FROM CarMaintenance cm2 JOIN Cars c ON cm2.CarID = c.CarID
WHERE c.CarType = 'Sedan');
```

Result Grid | Filter Rows:

CarID	MaintenanceCode	Cost
3	MAIN102	3000.00
2	MAIN103	5000.00
7	MAIN104	4000.00
5	MAIN105	2500.00
8	MAIN107	4500.00
6	MAIN108	6000.00
9	MAIN109	3500.00
10	MAIN110	8000.00

JOINS

1) INNER JOIN

Example: Get all bookings along with the customer name

```
SELECT b.BookingCode, c.FirstName, c.LastName, b.TotalAmount FROM Bookings b
INNER JOIN Customers c ON b.CustomerID = c.CustomerID;
```

Result Grid | Filter Rows: | Export:

BookingCode	FirstName	LastName	TotalAmount
BKG101	Rahul	Sharma	3600.00
BKG102	Sneha	Patil	4000.00
BKG103	Amit	Verma	4500.00
BKG104	Priya	Mehta	3400.00
BKG105	Karan	Joshi	3600.00
BKG106	Anjali	Kapoor	12000.00
BKG107	Vikram	Singh	6600.00
BKG108	Neha	Rao	3200.00
BKG109	Rohit	Nair	13500.00
BKG110	Meera	Desai	5600.00

2) OUTER JOIN

A) LEFT OUTER JOIN

Example: List all customers and their bookings (even if some customers haven't booked yet)

```
SELECT c.FirstName, c.LastName, b.BookingCode, b.TotalAmount
FROM Customers c
```

LEFT JOIN Bookings b ON c.CustomerID = b.CustomerID;

Result Grid | Filter Rows: | Export:

FirstName	LastName	BookingCode	TotalAmount
Rahul	Sharma	BKG101	3600.00
Sneha	Patil	BKG102	4000.00
Amit	Verma	BKG103	4500.00
Priya	Mehta	BKG104	3400.00
Karan	Joshi	BKG105	3600.00
Anjali	Kapoor	BKG106	12000.00
Vikram	Singh	BKG107	6600.00
Neha	Rao	BKG108	3200.00
Rohit	Nair	BKG109	13500.00
Meera	Desai	BKG110	5600.00
Arjun	Kulkarni	NULL	NULL
Simran	Kaur	NULL	NULL
Dev	Malhotra	NULL	NULL
Ishita	Roy	NULL	NULL
Manish	Choudhary	NULL	NULL

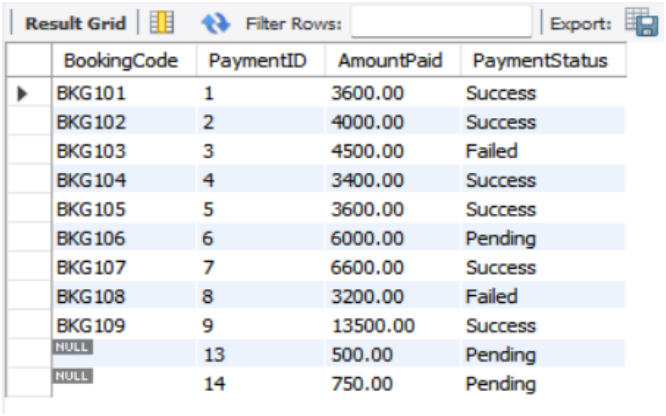
B) RIGHT OUTER JOIN

Example: List all bookings and any payment details (even if payment isn't done yet)

SELECT b.BookingCode, p.PaymentID, p.AmountPaid, p.PaymentStatus

FROM Bookings b

RIGHT JOIN Payments p ON b.BookingID = p.BookingID;



Result Grid | Filter Rows: | Export:

BookingCode	PaymentID	AmountPaid	PaymentStatus
BKG101	1	3600.00	Success
BKG102	2	4000.00	Success
BKG103	3	4500.00	Failed
BKG104	4	3400.00	Success
BKG105	5	3600.00	Success
BKG106	6	6000.00	Pending
BKG107	7	6600.00	Success
BKG108	8	3200.00	Failed
BKG109	9	13500.00	Success
NULL	13	500.00	Pending
NULL	14	750.00	Pending

C) FULL OUTER JOIN

Example: List all cars and all maintenance records, even if some cars have no maintenance and some maintenance has no car (for demonstration)

SELECT car.CarCode, cm.MaintenanceCode, cm.Cost FROM Cars car

LEFT JOIN CarMaintenance cm ON car.CarID = cm.CarID

UNION

SELECT car.CarCode, cm.MaintenanceCode, cm.Cost FROM Cars car

RIGHT JOIN CarMaintenance cm ON car.CarID = cm.CarID;

Result Grid | Filter Rows:

CarCode	MaintenanceCode	Cost
CAR101	MAIN101	1500.00
CAR102	MAIN103	5000.00
CAR103	MAIN102	3000.00
CAR104	MAIN106	1200.00
CAR105	MAIN105	2500.00
CAR106	MAIN108	6000.00
CAR107	MAIN104	4000.00
CAR108	MAIN107	4500.00
CAR109	MAIN109	3500.00
CAR110	MAIN110	8000.00

3) CROSS JOIN:

Example: Generate all possible employee-car pairs (for assigning mechanics)

SELECT e.FirstName, e.LastName, car.CarModel FROM Employees e

CROSS JOIN Cars car;

Result Grid | Filter Rows:

FirstName	LastName	CarModel
Arjun	Malhotra	Swift Dzire
Meena	Chopra	Swift Dzire
Suresh	Menon	Swift Dzire
Anita	Desai	Swift Dzire
Vikram	Kapoor	Swift Dzire
Kiran	Patil	Swift Dzire
Priya	Iyer	Swift Dzire
Aakash	Reddy	Swift Dzire
Suman	Verma	Swift Dzire
Rajesh	Sharma	Swift Dzire
Arjun	Malhotra	i20
Meena	Chopra	i20
Suresh	Menon	i20
Anita	Desai	i20
Vikram	Kapoor	i20
Kiran	Patil	i20

Result 35 ×

4) SELF JOIN:

Example: Find pairs of employees with the same role

SELECT e1.FirstName AS Employee1, e2.FirstName AS Employee2, e1.Role FROM Employees e1

INNER JOIN Employees e2 ON e1.Role = e2.Role AND e1.EmployeeID < e2.EmployeeID;

Result Grid | Filter Rows:

Employee1	Employee2	Role
Rajesh	Meena	Manager
Suman	Kiran	Staff
Suman	Suresh	Staff
Aakash	Vikram	Driver
Aakash	Arjun	Driver
Priya	Anita	Mechanic
Kiran	Suresh	Staff
Vikram	Arjun	Driver

WINDOWS FUNCTION

1) ROW NUMBER()

Example: Rank bookings by total amount per customer

SELECT CustomerID, BookingCode, TotalAmount, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC) AS BookingRank FROM Bookings ORDER BY CustomerID, BookingRank;

Result Grid | Filter Rows: | Export:

CustomerID	BookingCode	TotalAmount	BookingRank
1	BKG101	3600.00	1
2	BKG102	4000.00	1
3	BKG103	4500.00	1
4	BKG104	3400.00	1
5	BKG105	3600.00	1
6	BKG106	12000.00	1
7	BKG107	6600.00	1
8	BKG108	3200.00	1
9	BKG109	13500.00	1
10	BKG110	5600.00	1

2) RANK()

Example: Rank cars by number of bookings

SELECT CarID, TotalBookings, RANK() OVER (ORDER BY TotalBookings DESC) AS BookingRank

FROM (SELECT CarID, COUNT(BookingID) AS TotalBookings FROM Bookings GROUP BY CarID) AS CarBookingCounts;

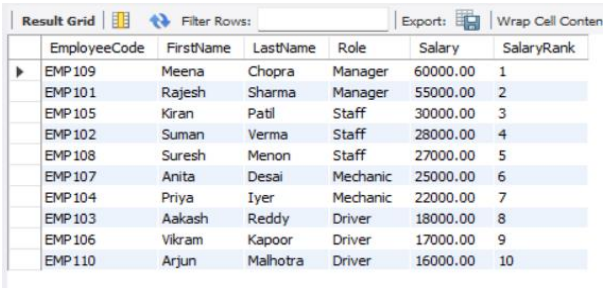
Result Grid | Filter Rows:

CarID	TotalBookings	BookingRank
1	1	1
2	1	1
3	1	1
4	1	1
5	1	1
6	1	1
7	1	1
8	1	1
9	1	1
10	1	1

3) DENSE_RANK()

Example: Rank employees by their salary, but if two employees have the same salary, they get the same rank without leaving gaps

SELECT EmployeeCode, FirstName, LastName,Role, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS SalaryRank FROM Employees;



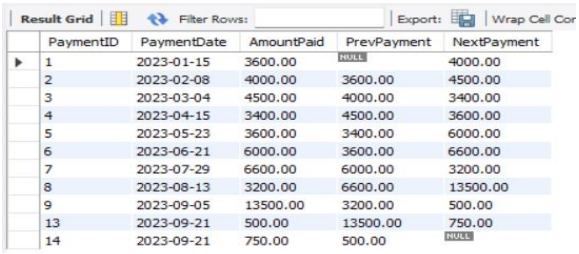
EmployeeCode	FirstName	LastName	Role	Salary	SalaryRank
EMP109	Meena	Chopra	Manager	60000.00	1
EMP101	Rajesh	Sharma	Manager	55000.00	2
EMP105	Kiran	Patil	Staff	30000.00	3
EMP102	Suman	Verma	Staff	28000.00	4
EMP108	Suresh	Menon	Staff	27000.00	5
EMP107	Anita	Desai	Mechanic	25000.00	6
EMP104	Priya	Iyer	Mechanic	22000.00	7
EMP103	Aakash	Reddy	Driver	18000.00	8
EMP106	Vikram	Kapoor	Driver	17000.00	9
EMP110	Arjun	Malhotra	Driver	16000.00	10

4) LAG() & LEAD()

Example: Compare each payment with the previous payment

SELECT PaymentID, PaymentDate, AmountPaid, LAG(AmountPaid) OVER (ORDER BY PaymentDate) AS PrevPayment, LEAD(AmountPaid) OVER (ORDER BY PaymentDate) AS NextPayment

FROM Payments;



PaymentID	PaymentDate	AmountPaid	PrevPayment	NextPayment
1	2023-01-15	3600.00	NULL	4000.00
2	2023-02-08	4000.00	3600.00	4500.00
3	2023-03-04	4500.00	4000.00	3400.00
4	2023-04-15	3400.00	4500.00	3600.00
5	2023-05-23	3600.00	3400.00	6000.00
6	2023-06-21	6000.00	3600.00	6600.00
7	2023-07-29	6600.00	6000.00	3200.00
8	2023-08-13	3200.00	6600.00	13500.00
9	2023-09-05	13500.00	3200.00	500.00
13	2023-09-21	500.00	13500.00	750.00
14	2023-09-21	750.00	500.00	NULL

LAG → previous row's amount

LEAD → next row's amount

VIEW

1) Using FIRST_VALUE() in a View

Example: Show the first booking (lowest amount) per customer

CREATE VIEW FirstBookingPerCustomer AS SELECT CustomerID, BookingCode, TotalAmount, FIRST_VALUE(BookingCode) OVER (PARTITION BY CustomerID ORDER BY TotalAmount ASC) AS FirstBooking FROM Bookings;

SELECT * FROM FirstBookingPerCustomer;

Result Grid | Filter Rows: | Export:

CustomerID	BookingCode	TotalAmount	FirstBooking
1	BKG101	3600.00	BKG101
2	BKG102	4000.00	BKG102
3	BKG103	4500.00	BKG103
4	BKG104	3400.00	BKG104
5	BKG105	3600.00	BKG105
6	BKG106	12000.00	BKG106
7	BKG107	6600.00	BKG107
8	BKG108	3200.00	BKG108
9	BKG109	13500.00	BKG109
10	BKG110	5600.00	BKG110

2) Using LAST_VALUE() in a View

Example: Show the last booking (highest amount) per customer

CREATE VIEW LastBookingPerCustomer AS SELECT CustomerID, BookingCode, TotalAmount, LAST_VALUE(BookingCode) OVER (PARTITION BY CustomerID ORDER BY TotalAmount ASC RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS LastBooking FROM Bookings;

SELECT * FROM LastBookingPerCustomer;

Result Grid | Filter Rows: | Export:

CustomerID	BookingCode	TotalAmount	LastBooking
1	BKG101	3600.00	BKG101
2	BKG102	4000.00	BKG102
3	BKG103	4500.00	BKG103
4	BKG104	3400.00	BKG104
5	BKG105	3600.00	BKG105
6	BKG106	12000.00	BKG106
7	BKG107	6600.00	BKG107
8	BKG108	3200.00	BKG108
9	BKG109	13500.00	BKG109
10	BKG110	5600.00	BKG110

3) Using NTH_VALUE() in a View

Example: Show the 2nd highest booking amount per customer

CREATE VIEW SecondHighestBooking AS SELECT CustomerID, BookingCode, TotalAmount, NTH_VALUE(BookingCode, 2) OVER (PARTITION BY CustomerID ORDER BY TotalAmount DESC RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS SecondBooking FROM Bookings;

SELECT * FROM SecondHighestBooking;

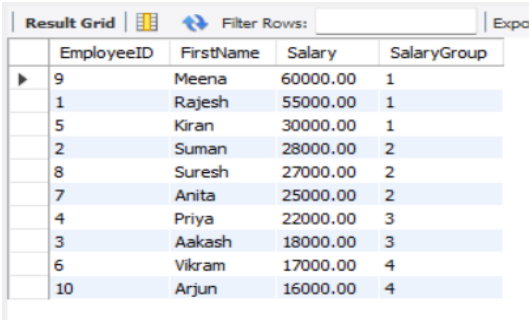
Result Grid | Filter Rows: | Export:

CustomerID	BookingCode	TotalAmount	SecondBooking
1	BKG101	3600.00	NULL
2	BKG102	4000.00	NULL
3	BKG103	4500.00	NULL
4	BKG104	3400.00	NULL
5	BKG105	3600.00	NULL
6	BKG106	12000.00	NULL
7	BKG107	6600.00	NULL
8	BKG108	3200.00	NULL
9	BKG109	13500.00	NULL
10	BKG110	5600.00	NULL

4) NTILE

Example: Divide employees into 4 salary groups (quartiles)

SELECT EmployeeID, FirstName, Salary, NTILE(4) OVER (ORDER BY Salary DESC) AS SalaryGroup
FROM Employees;



Result Grid | Filter Rows: | Expo

EmployeeID	FirstName	Salary	SalaryGroup
9	Meena	60000.00	1
1	Rajesh	55000.00	1
5	Kiran	30000.00	1
2	Suman	28000.00	2
8	Suresh	27000.00	2
7	Anita	25000.00	2
4	Priya	22000.00	3
3	Aakash	18000.00	3
6	Vikram	17000.00	4
10	Arjun	16000.00	4